

Multimodal Generation I

Large Foundational Models and Physical World Models for Multimodal AI

Lecture 5

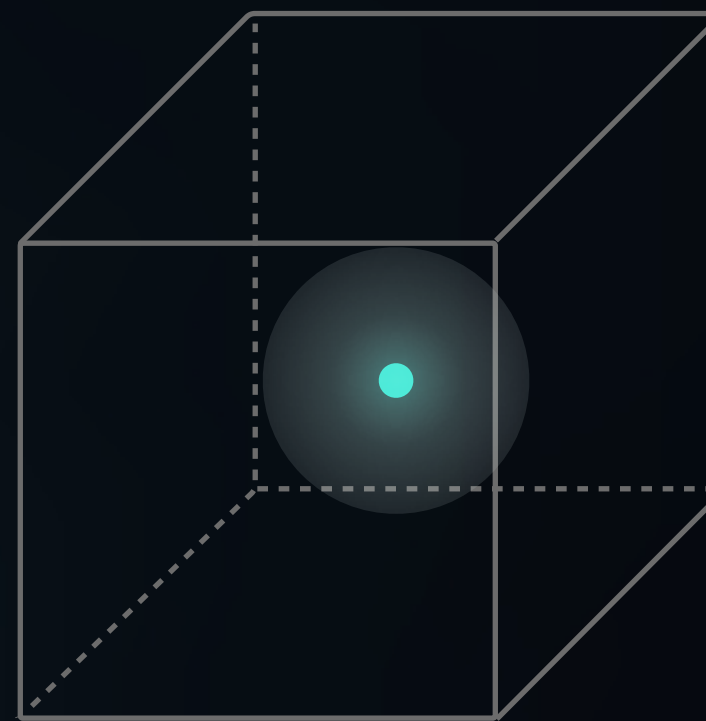
Jin-Hwa Kim and Sangdoo Yun

Multimodal Generation series

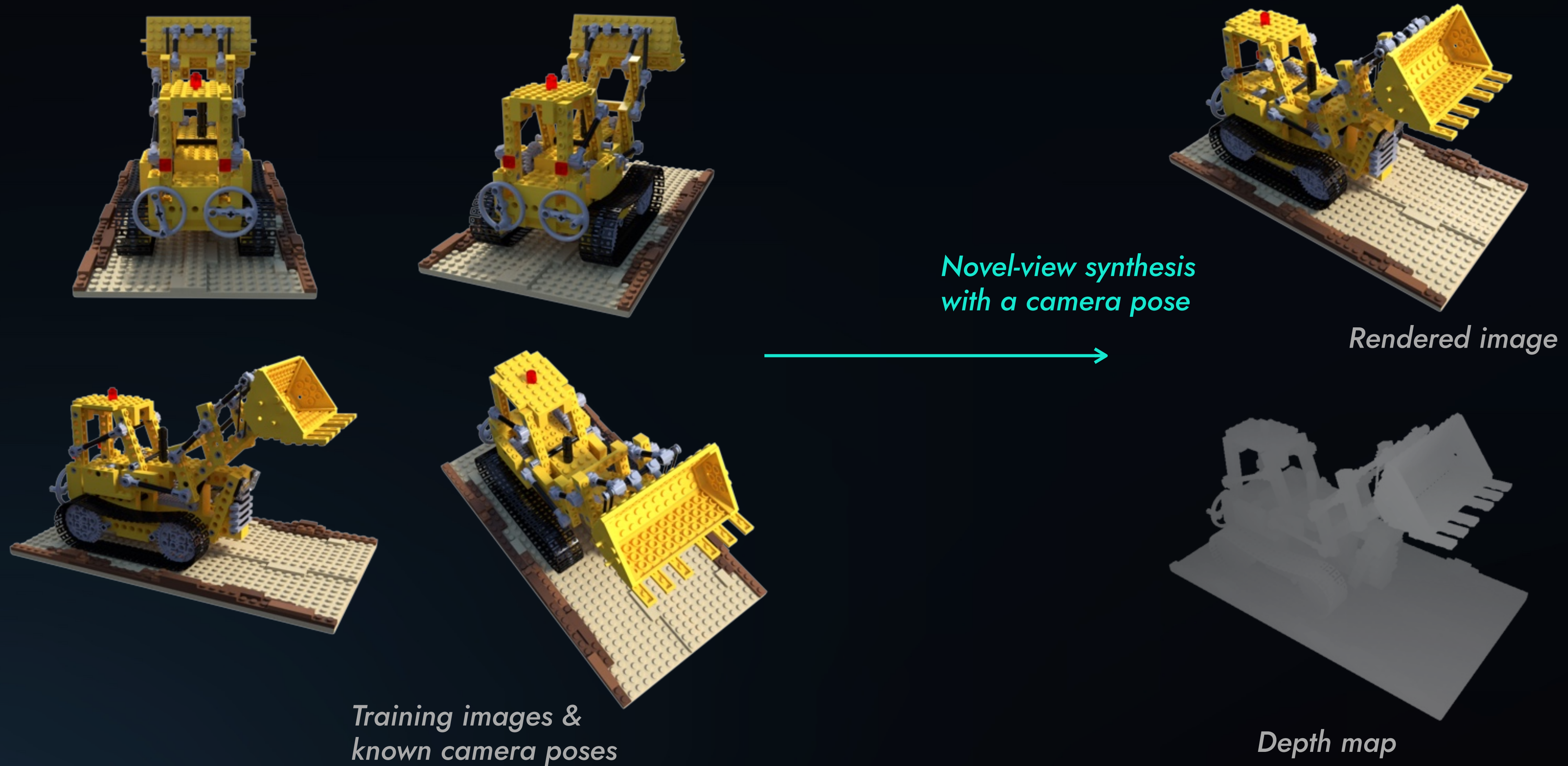
- *Multimodal Generation I, today*
 - NeRF and pinhole camera system
- *Multimodal Generation II on Oct 9 (Online)*
 - Topics in Neural Graphics: Camera pose refinement, Hash Grid, and surface modelings
- *Multimodal Generation III on Oct 16*
 - 3D generation using SDS, world models, and Seoul World Model

Neural radiance fields

- Neural radiance fields (NeRFs) represent a scene by neural networks and synthesize novel views ([Mildenhall et al., 2020](#)).
- The *radiance* means “light or heat as emitted or reflected by something.”
- The NeRFs is a learning framework for the *optics*.

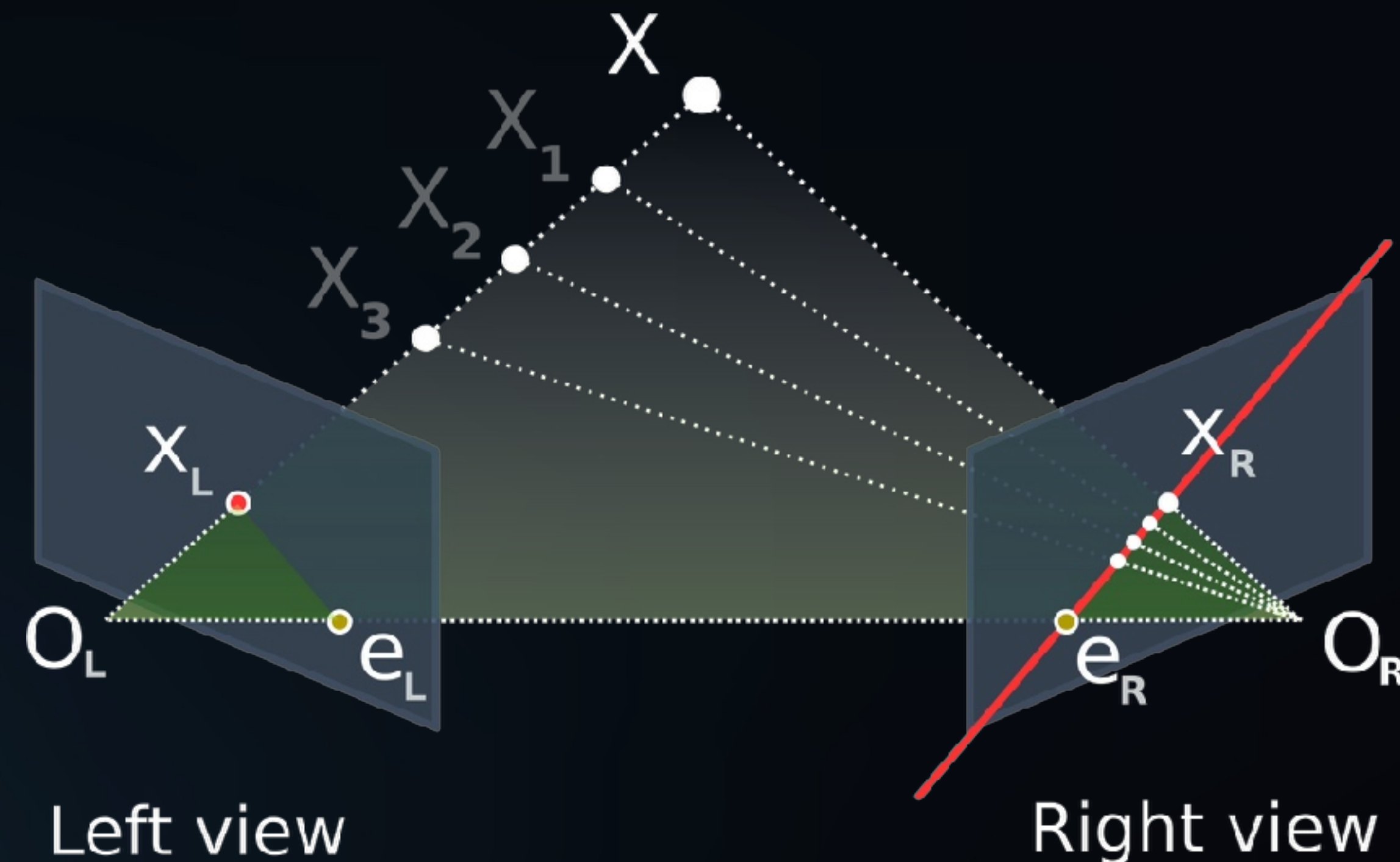


Novel-view synthesis



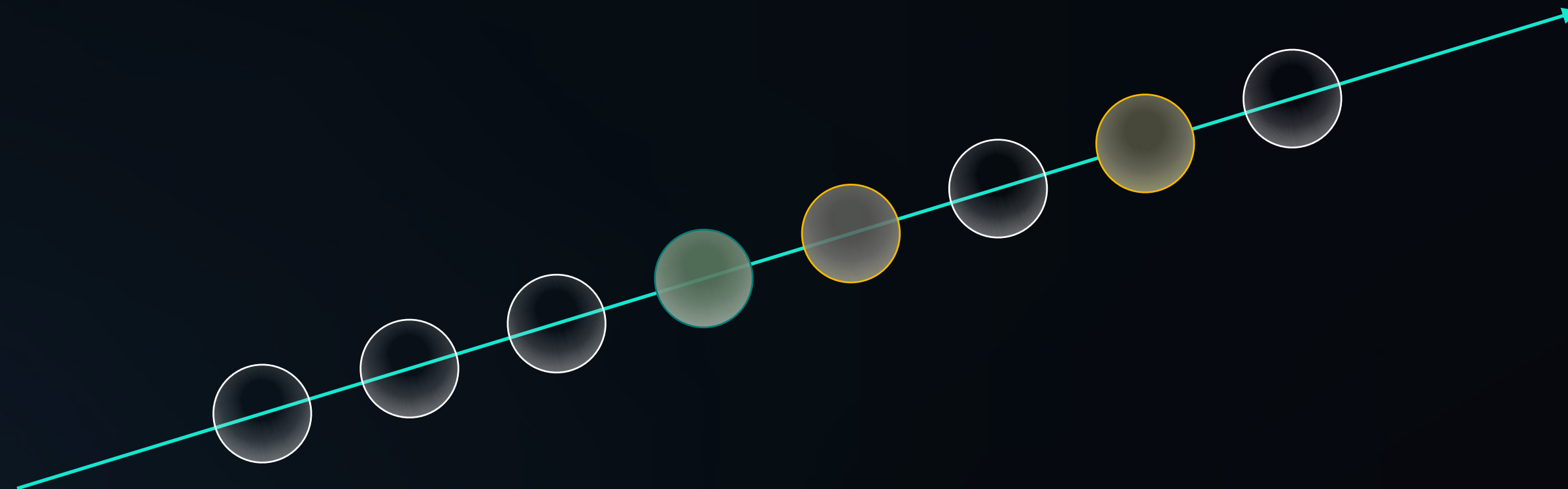
Epipolar constraint

- A geometric rule in computer vision stating that a point in one camera image must lie on a specific line—called an epipolar line—in a second camera image



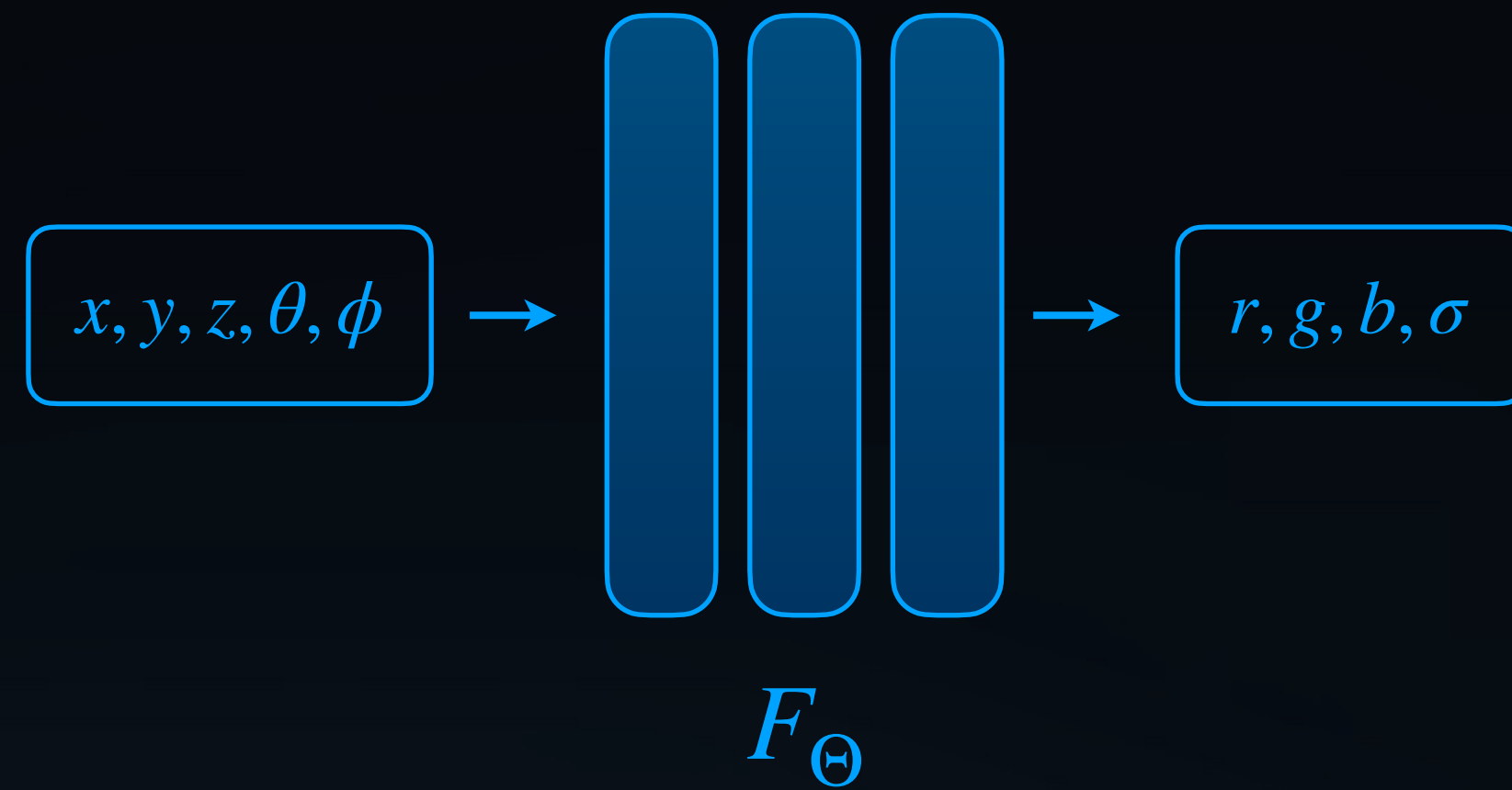
Radiance fields

- In 3D computer graphics, a *voxel* represents a value on a regular grid in continuous three-dimensional space.
- However, NeRFs represent each evaluated point using MLPs with two attributes: volume density σ and emitted color c . *That's why we refer to it as radiance!*



The term “voxel” is a compound word combining volume and pixel.

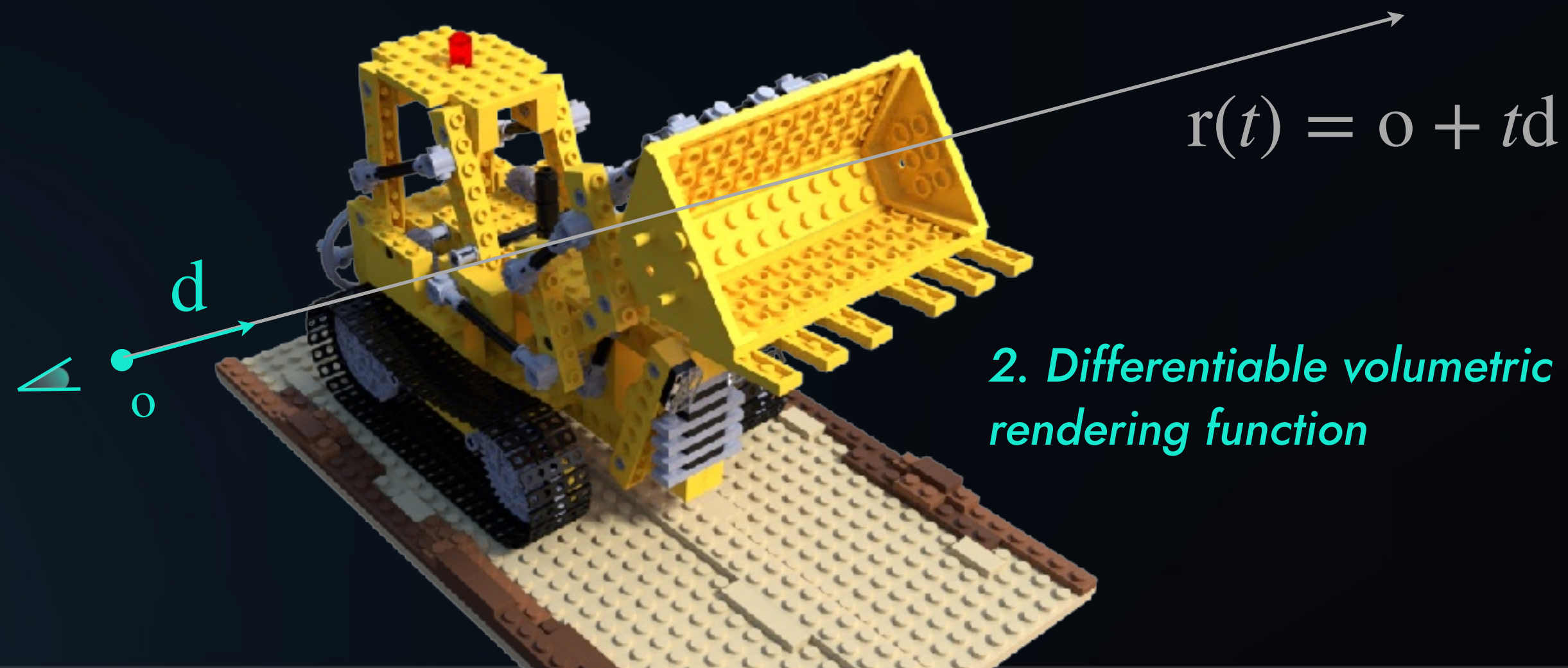
Three components



1. Neural volumetric 3D scene representation

$$\mathcal{L} = \sum_{\mathbf{r} \in \mathcal{R}} \|\hat{C}(\mathbf{r}) - C(\mathbf{r})\|_2^2$$

3. Optimization to synthesize all training images



2. Differentiable volumetric rendering function

Demonstration

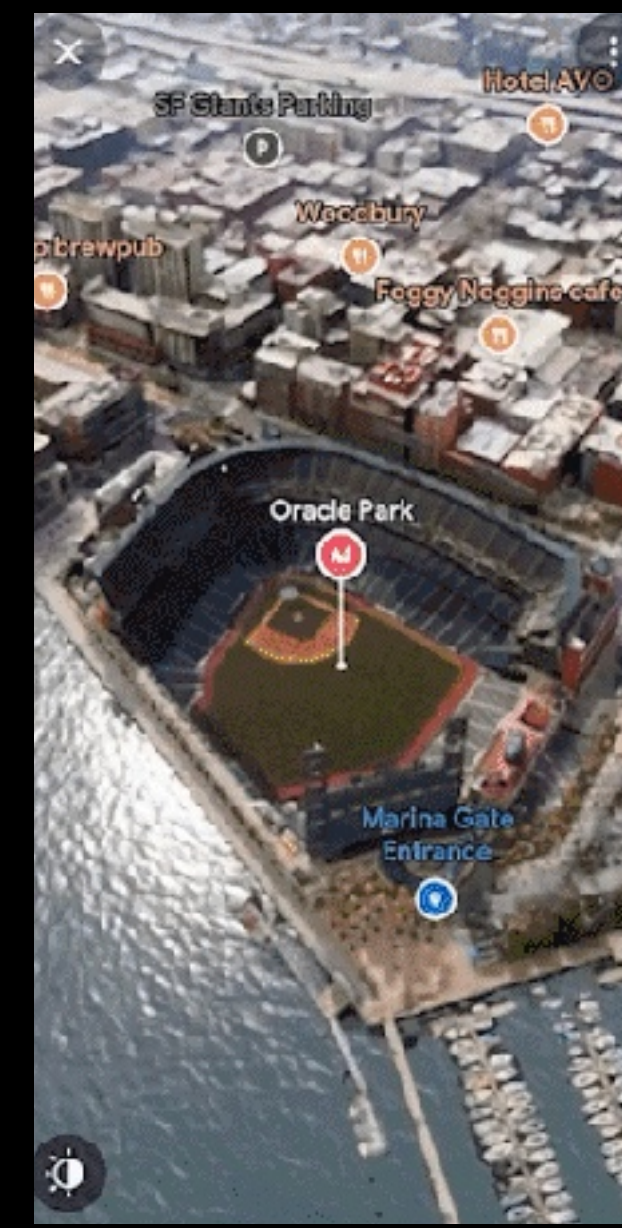
- A vanilla NeRF can give ...



Commercial products
using NeRFs and classical methods

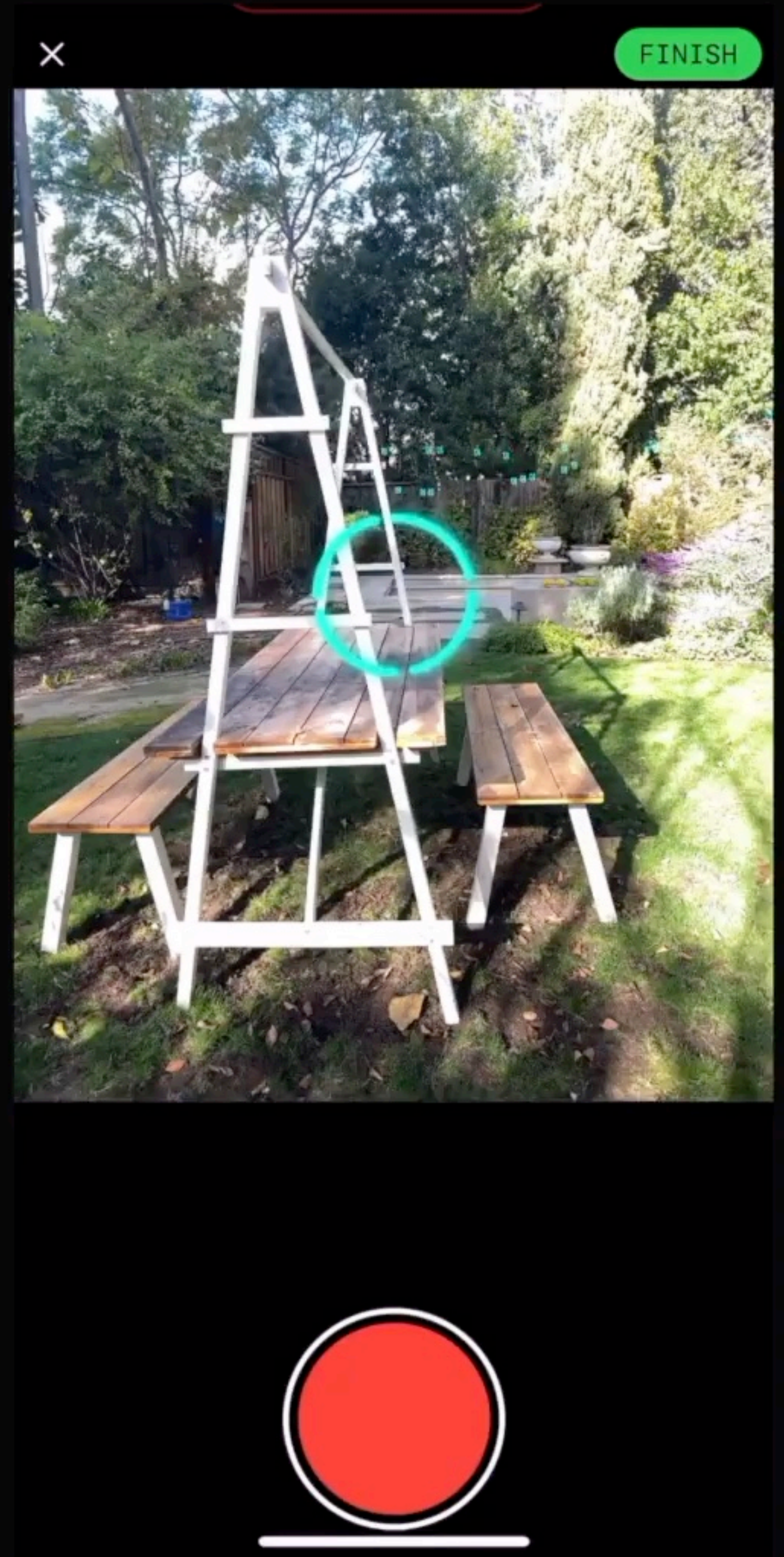
Google Maps

- “New updates that make Maps look and feel more like the real world” —Sep 28, 2022
- Google Maps help you get the feel for a neighborhood before you go, explore **250+ landmarks in aerial view**, search for **nearby places with Live View** and more.
- “Digital twin”

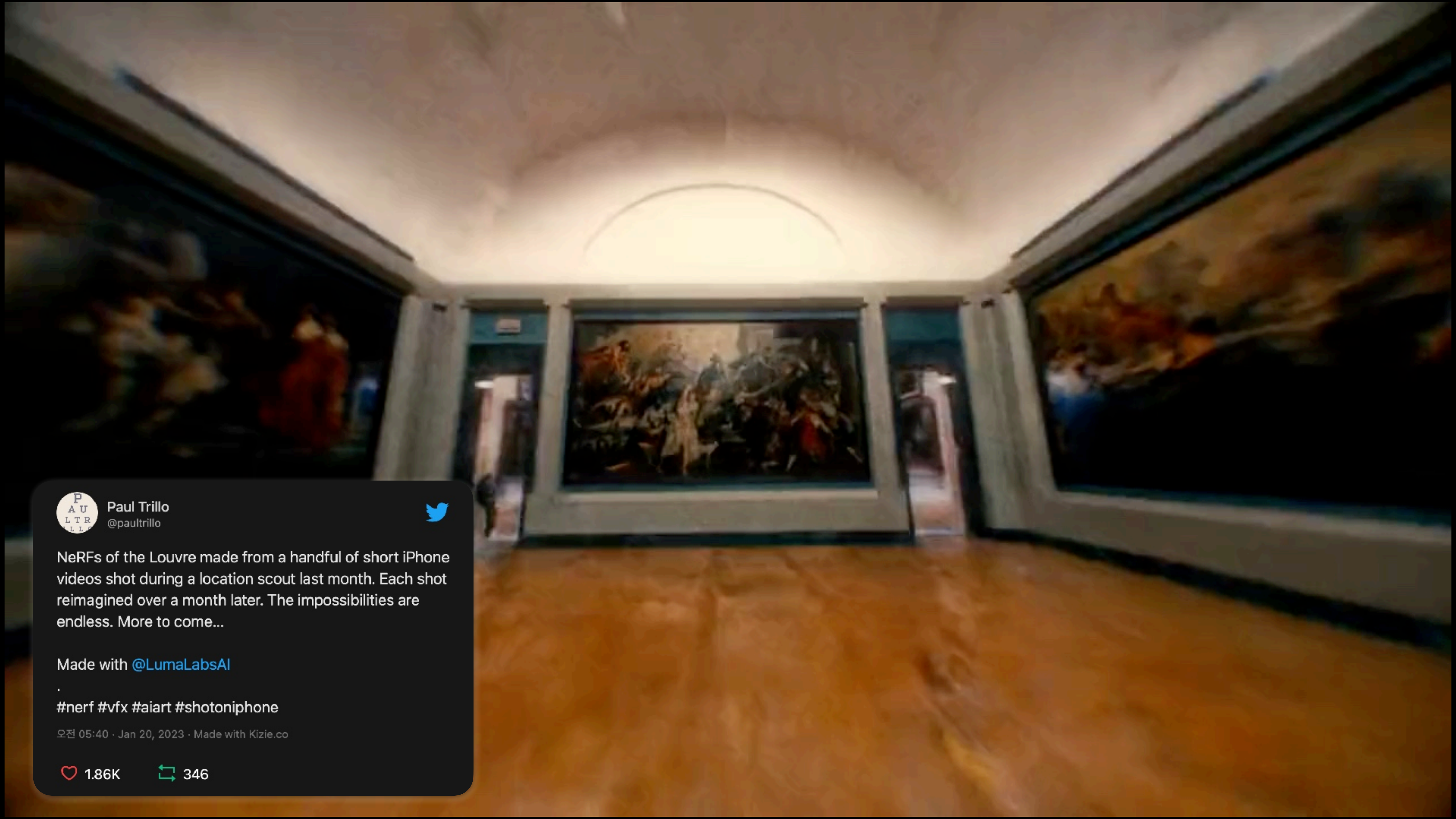




Brandenburg Gate, Germany



Movie credit: Luma AI at <https://lumalabs.ai/>



Paul Trillo
@paultrillo



NeRFs of the Louvre made from a handful of short iPhone videos shot during a location scout last month. Each shot reimagined over a month later. The impossibilities are endless. More to come...

Made with [@LumaLabsAI](#)

#nerf #vfx #aiart #shotoniphone

오전 05:40 · Jan 20, 2023 · Made with Kizie.co



1.86K



346



Balenciaga Track Sneaker

White Mesh and Nylon

<https://www.balenciaga.com/en-kr/track-sneaker-white-542023W1GB19000.html>

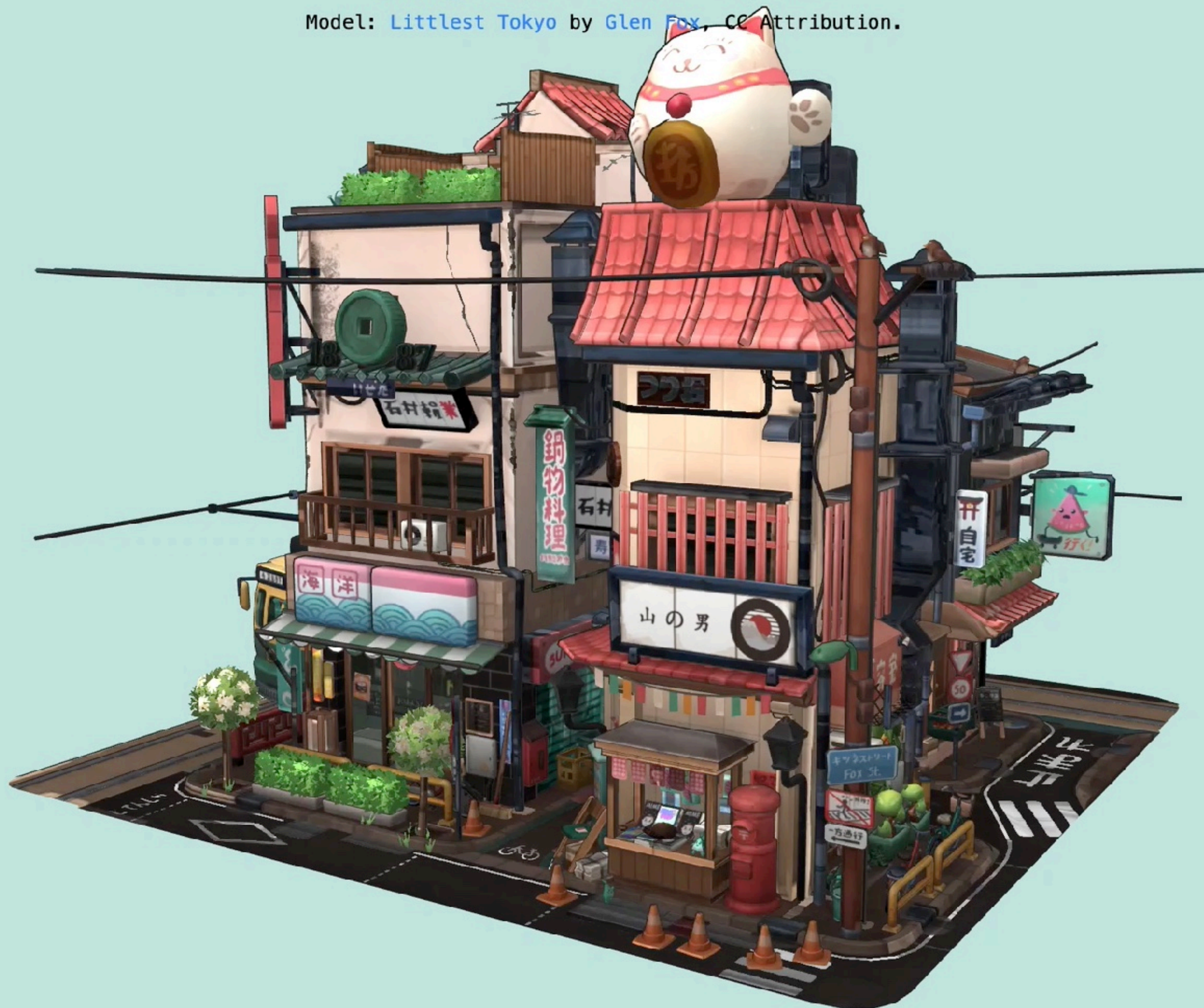


GT



Ours

Our customized model with PSNR of 35.35 and SSIM of 0.984.



NAVER
LABS

3D City Models
of Seoul





The Vegetarian

Bomi Kim, 2025

Acrylic on canvas

66.0 × 59.0 cm

Collection of Jin-Hwa Kim.



Reality Converter by Apple

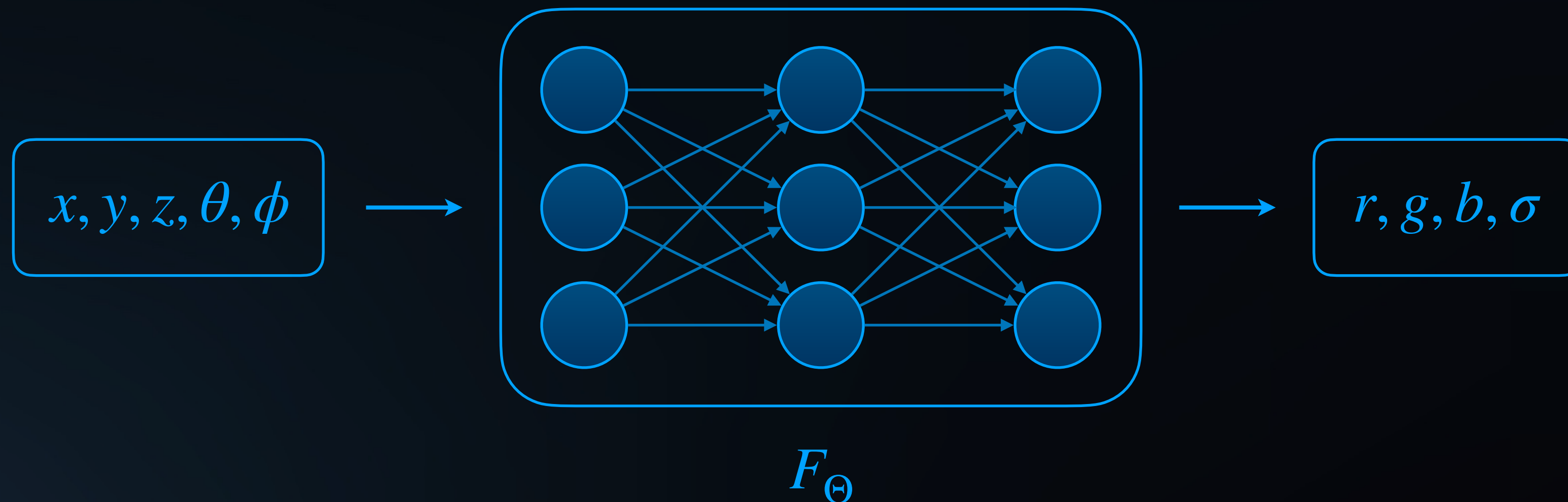
*Makes it easy to convert, view, and customize USDZ 3D objects on Mac.
Customize material properties with your own textures, and edit file metadata.
You can even preview your USDZ object under a variety of lighting and
environmental conditions*



Volume renderings

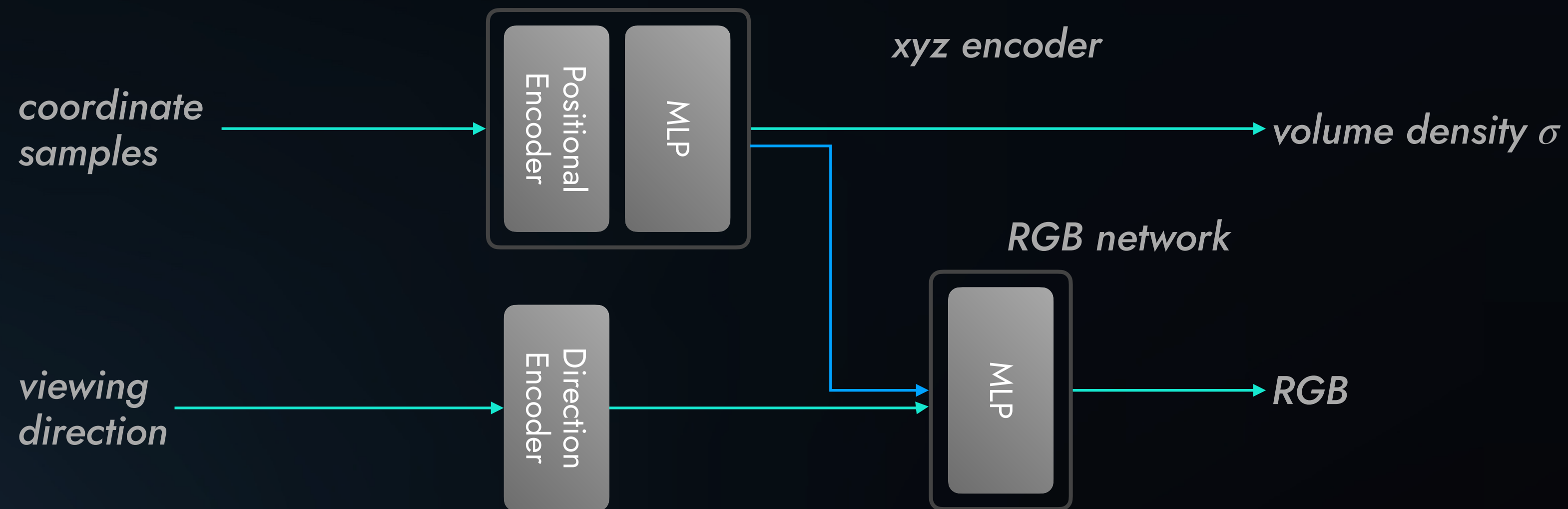
NeRF scene representation

- An MLP network $F_{\Theta} : (x, d) \rightarrow (c, \sigma)$ takes 3D location $x = (x, y, z)$ and 2D viewing direction $d = (\theta, \phi)$, giving an emitted color $c = (r, g, b)$ and volume density σ .
- A view-dependent modeling for physical world



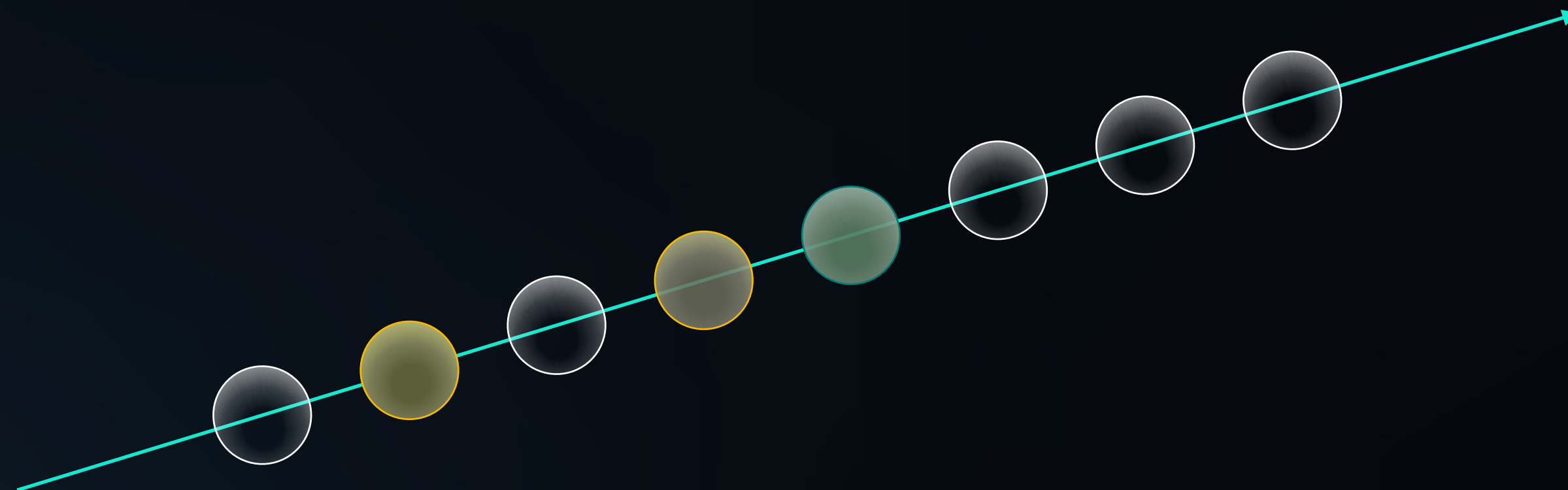
Non-Lambertian NeRF

- The emitted color depends on *both* location and viewing direction.
- While, the brightness of a Lambertian surface to an observer is the same regardless of the viewing direction. The surface's luminance is isotropic, and the luminous intensity obeys Lambert's cosine law. *Yes, NeRF's non-Lambertian.*



Volume rendering with radiance fields

- Classical volume rendering ([Kajiya et al., 1984](#))
- Numerical estimation using quadrature
- Hierarchical volume sampling



Simplification

- The volume rendering used in NeRF does not consider *scattering* for computational efficiency, although it considers *absorption* and *emission*.



Absorption



Emission



Scattering

Simplification

- The volume rendering used in NeRF does not consider *scattering* for computational efficiency, although it considers *absorption* and *emission*.



Absorption



Emission

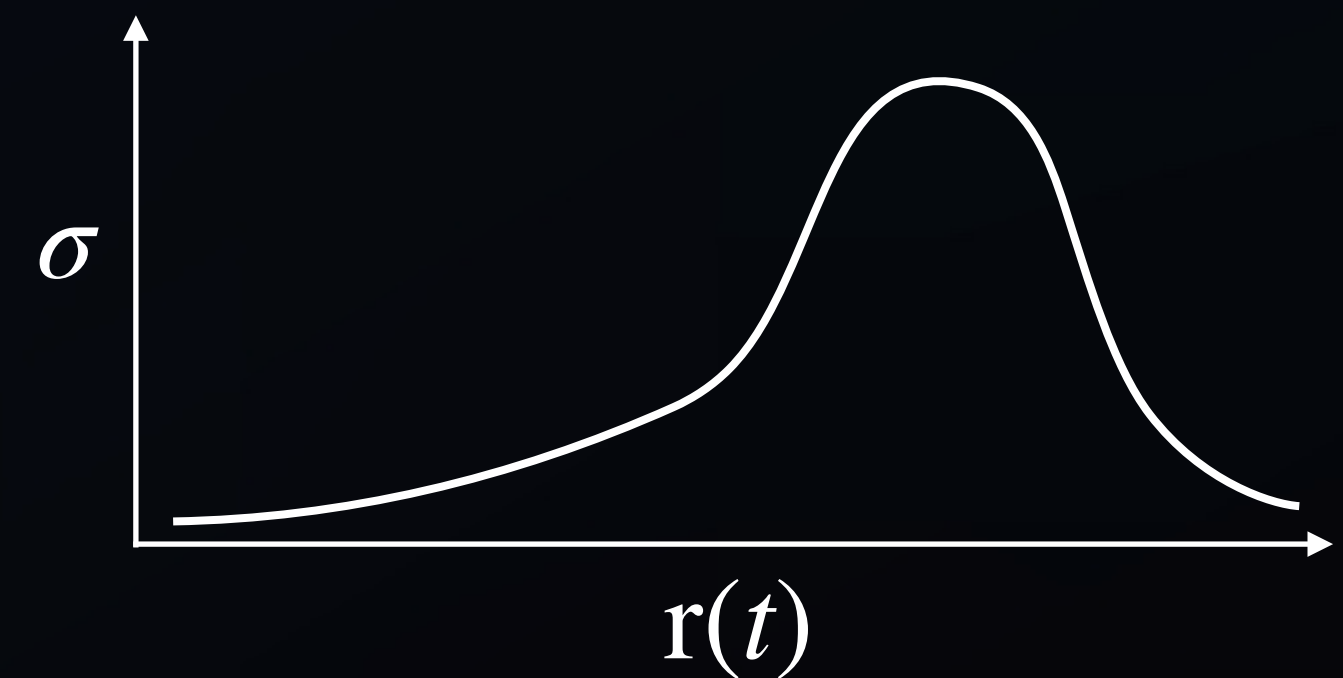
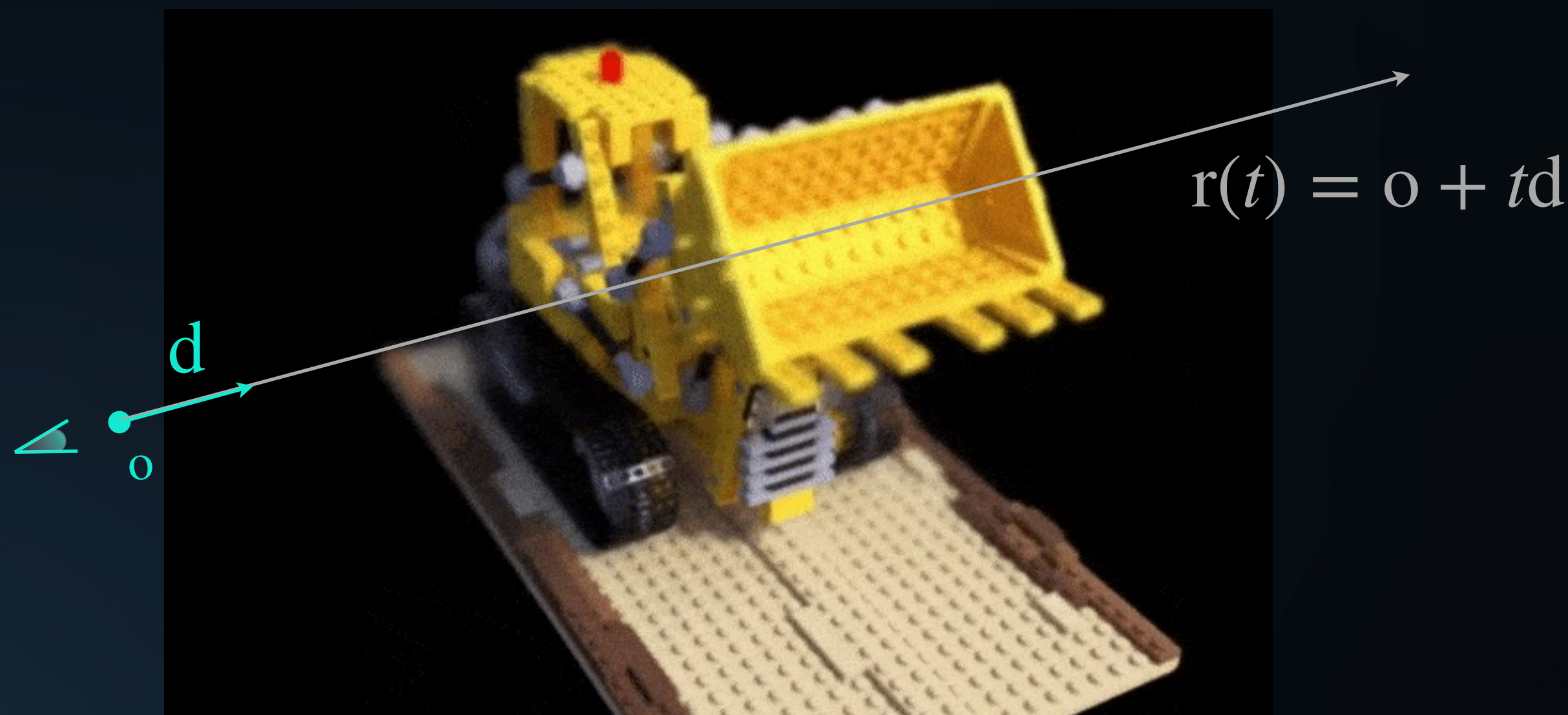


Scattering

Classical volume rendering

- The volume density $\sigma(x)$ is the differential probability at an infinitesimal particle at x .
- The expected color $C(r)$ of camera ray $r(t) = o + td$ with near and far bounds t_n and t_f , respectively, is as follows:

$$C(r) = \int_{t_n}^{t_f} T(t) \sigma(r(t)) c(r(t), d) dt, \text{ where } T(t) = \exp\left(-\int_{t_n}^t \sigma(r(s)) ds\right)$$



Classical volume rendering

- The volume density $\sigma(x)$ is the differential probability at an infinitesimal particle at x .
- The expected color $C(r)$ of camera ray $r(t) = o + td$ with near and far bounds t_n and t_f , respectively, is as follows:

$$C(r) = \int_{t_n}^{t_f} T(t) \sigma(r(t)) c(r(t), d) dt, \text{ where } T(t) = \exp\left(-\int_{t_n}^t \sigma(r(s)) ds\right)$$

- $T(t)$ is the *accumulated transmittance* along the ray r , i.e., the *probability* that the ray travels from t_n to t without hitting any other particle.

Transmittance

- The transmittance function $T(t)$ is the probability of a ray traveling over the interval $[0, t)$ without hitting any particles. $T(t + dt)$ is equal to $T(t)(1 - dt \cdot \sigma(t))$.

$$T(t + dt) = T(t)(1 - dt \cdot \sigma(t))$$

A hitting probability is the density.

$$\frac{T(t + dt) - T(t)}{dt} = T'(t) = -T(t)\sigma(t)$$

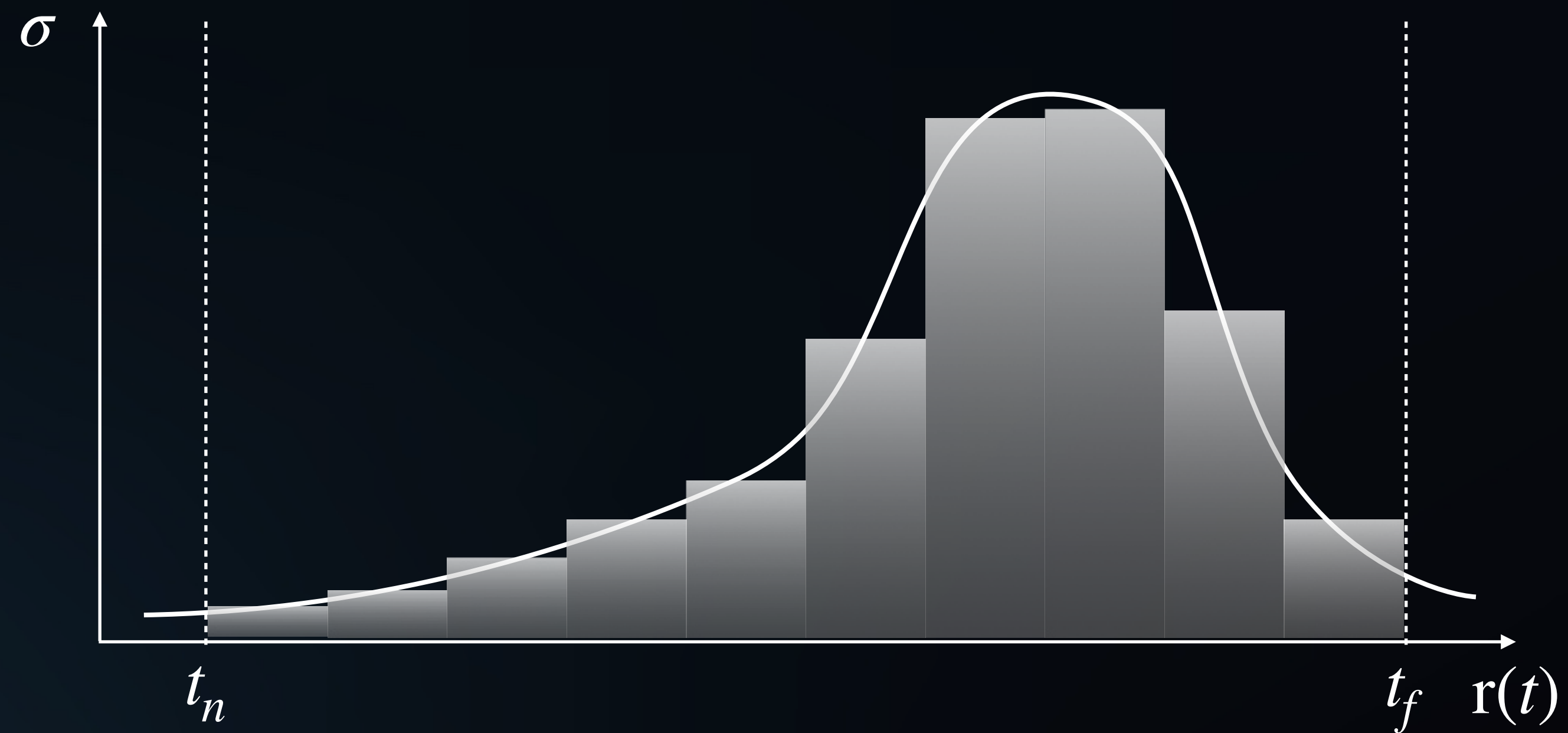
Ref. logarithmic
differentiation

$$\frac{T(t)'}{T(t)} = -\sigma(t) \Leftrightarrow \int_a^b \frac{T(t)'}{T(t)} dt = \log T(t) \Big|_a^b = -\int_a^b \sigma(t) dt$$

$$\log T(b) - \log T(a) = -\int_a^b \sigma(t) dt \Leftrightarrow \frac{T(b)}{T(a)} = \exp\left(-\int_a^b \sigma(t) dt\right)$$

Numerical estimation using quadrature

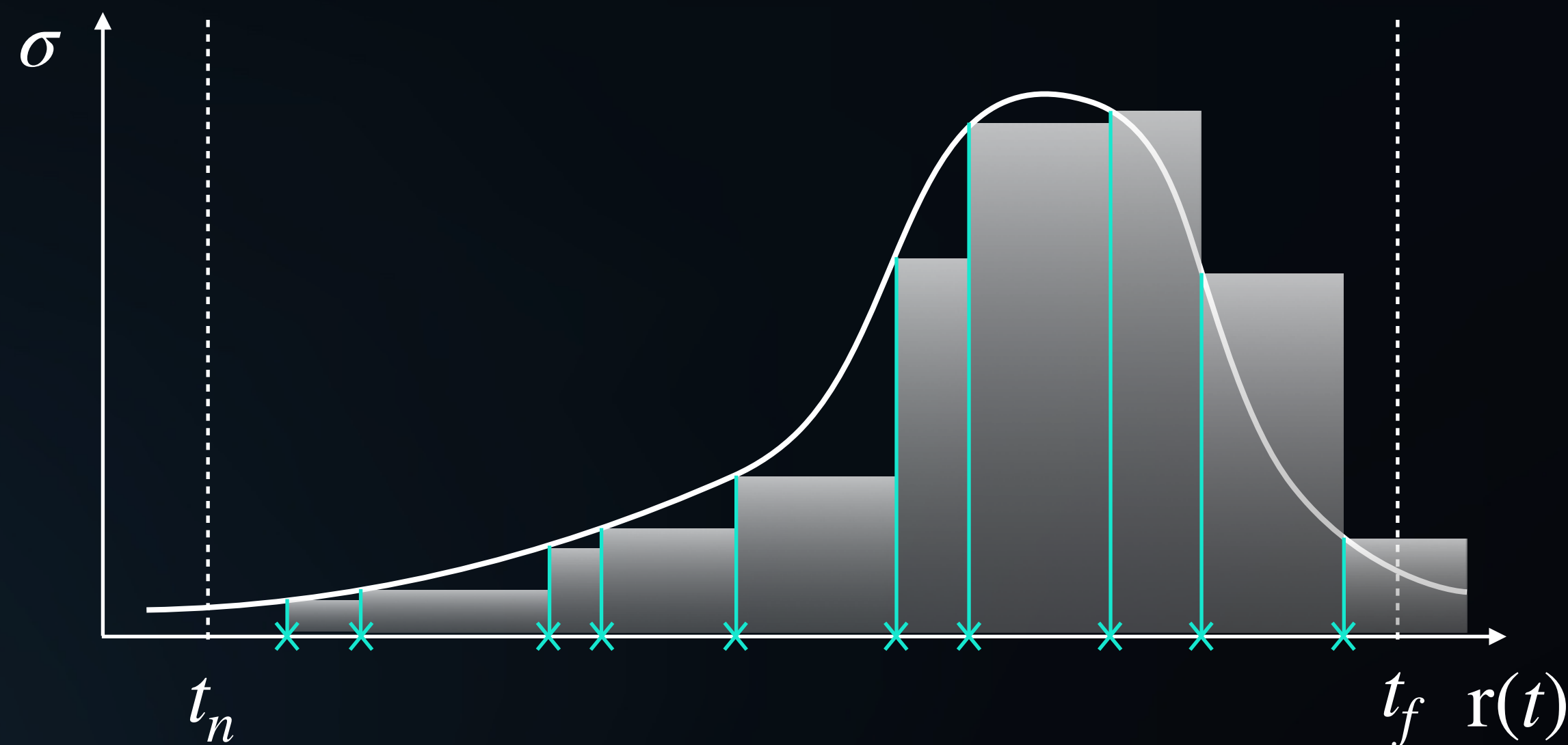
- The exact calculation of the integral is intractable and inefficient. One approximation is to estimate using quadrature, *compensating representation's resolution*.



Stratified sampling

- Stratified sampling partition $[t_n, t_f]$ into N evenly-spaced bins, and draw a sample from each bin, uniformly at random. Totally, N samples will be drawn.

$$t_i \sim \mathcal{U}\left[t_n + \frac{i-1}{N}(t_f - t_n), t_n + \frac{i}{N}(t_f - t_n)\right] \text{ for } i \in [1, 2, \dots, N]$$



Stratified sampling

- Stratified sampling partition $[t_n, t_f]$ into N evenly-spaced bins, and draw a sample from each bin, uniformly at random. Totally, N samples will be drawn.

$$t_i \sim \mathcal{U} \left[t_n + \frac{i-1}{N}(t_f - t_n), t_n + \frac{i}{N}(t_f - t_n) \right] \text{ for } i \in [1, 2, \dots, N]$$

- The volume rendering with the *stratified sampling* gives the estimate $C(r)$.

$$\hat{C}(r) = \sum_{i=1}^N T_i \left(1 - \exp(-\sigma_i \delta_i) \right) c_i \text{ where } T_i = \exp \left(- \sum_{j=1}^{i-1} \sigma_j \delta_j \right)$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples, and σ_i and c_i are the volume density and colors at t_i , which is cheap to evaluate using MLPs.

Stratified sampling

- The volume rendering with the *stratified sampling* gives the estimate $C(r)$.

$$\hat{C}(r) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) c_i \quad \text{where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples, and σ_i and c_i are the volume density and colors at t_i , which is cheap to evaluate using MLPs.

- **Proof.** Split the interval into n small pieces of length δ/n . The probability of passing through each piece is $1 - \sigma\delta/n$. Since the ray must pass through all of them, we multiply:

$$\left(1 - \frac{\sigma\delta}{n}\right)^n \xrightarrow{n \rightarrow \infty} \exp(-\sigma\delta)$$

- So the hit probability over the finite interval is $1 - \exp(-\sigma\delta)$.

Hierarchical volume sampling

- An adaptive sampling strategy with two networks: the *coarse* and *fine*.
- The stratified sampling is for the coarse network, while a more informed sampling is for the fine network using the previous results.
- Then, the alpha-composited color and a piecewise constant PDF from this coarse network are the ground for the hierarchical volume sampling.

$$\hat{C}_c(\mathbf{r}) = \sum_{i=1}^{N_c} w_i c_i, \quad w_i = T_i(1 - \exp(-\sigma_i \delta_i))$$

- $\hat{C}_c(\mathbf{r})$ enables a faster evaluation of a rendered color, while the normalized w_i , $\hat{w}_i = w_i / \sum_{j=1}^{N_c} w_j$, defines the PDF for the sampling for fine networks.

Hierarchical volume sampling (Cont'd)

- Then, compute the final rendered color of the ray $\hat{C}_f(r)$ using $N_c + N_f$ samples.
- Similarly to importance sampling, but it is a non-uniform discretization rather than an independent probabilistic estimation of the entire integral.

Positional encoding

- Spectral bias (Tancik et al., 2020) may interfere the learning of NeRF performing poorly at high-frequency variation in color and geometry (Rahaman et al., 2018).

- Formulating with a composition of two functions $F_{\Theta} = F'_{\Theta} \circ \gamma$ where $\gamma : \mathbb{R} \rightarrow \mathbb{R}^{2L}$

$$\gamma(p) = (\sin(2^0\pi p), \cos(2^0\pi p), \dots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p))$$

- Applied to each of the three coordinate values in $\mathbf{x} \in [-1, 1]^3$.
- It is known that it makes MLP easily approximate a higher frequency function.

Optimization

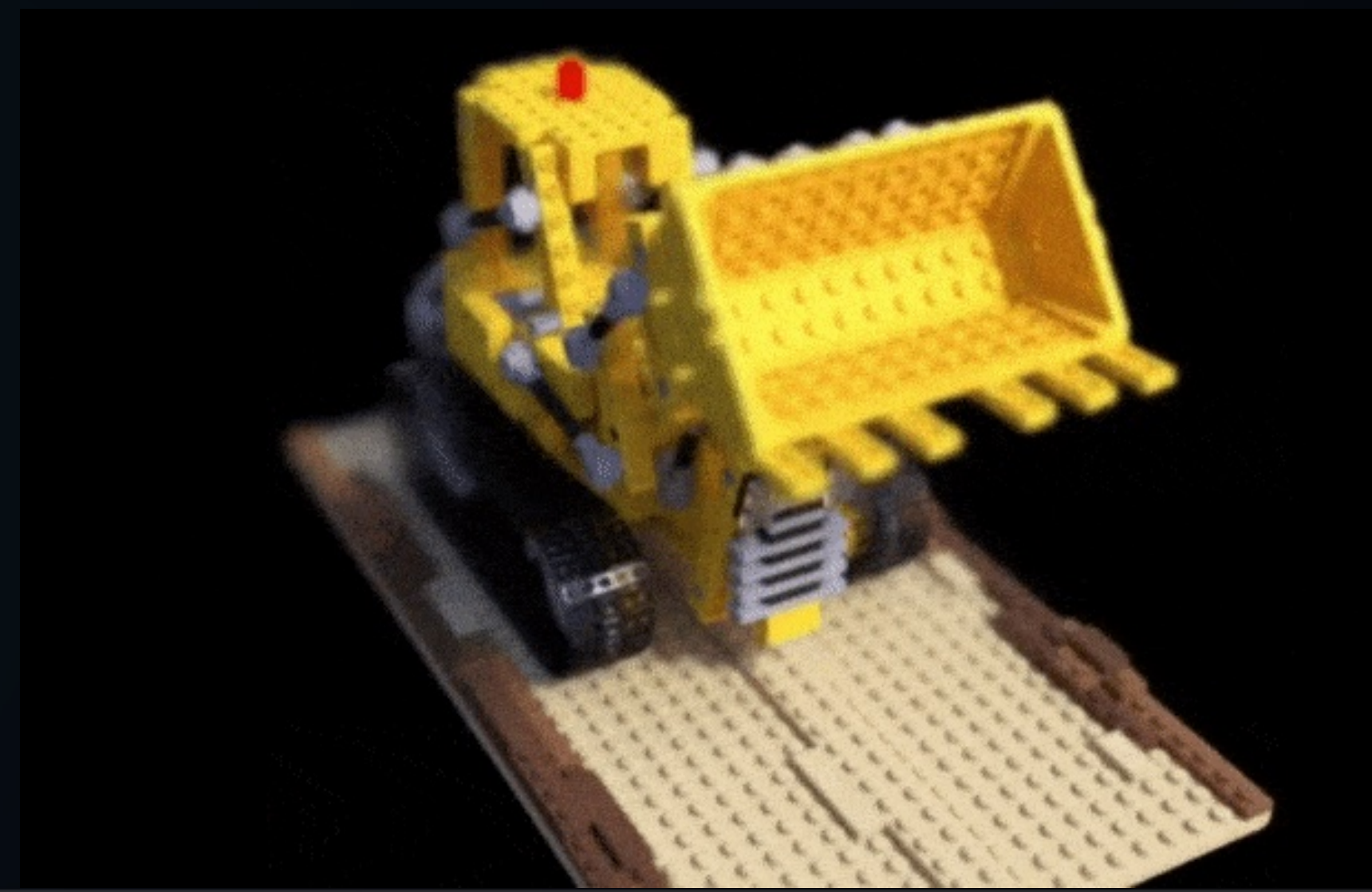
- Assume that we know ground-truth camera poses and intrinsics, and scene bounds. The COLMAP structure-from-motion (SfM) package provides a good initialization for them (Schönberger et al., 2016).
- The objective function is the total squared error between the rendered and true pixel colors for both the coarse and fine renderings:

$$\mathcal{L} = \sum_{\mathbf{r} \in \mathcal{R}} \left[\|\hat{C}_c(\mathbf{r}) - C(\mathbf{r})\|_2^2 + \|\hat{C}_f(\mathbf{r}) - C(\mathbf{r})\|_2^2 \right]$$

- where $\mathcal{R}$ is the randomly sampled batch of camera rays from the set of all pixels in the dataset. The first term ensures that the estimation of coarse network is reliable during training.

Datasets

- NeRF-Synthesis ([Mildenhall et al., 2020](#))
 - Originally, this is the third set of Realistic Synthetic 360° consists of *eight* objects of complicated geometry and realistic non-Labertian materials.
 - *Six* of them are rendered from the viewpoints on the *upper hemisphere*, while *the others* are from the viewpoints on a *full sphere* at 800x800 resolution.
 - 100 training views and 200 for testing
 - Drum, ficus, hotdog, lego, mic, etc.



Datasets (Cont'd)

- LLFF ([Mildenhall et al., 2019, 2020](#))
 - Real images of complex scenes
 - Roughly *forward-facing* images of 8 scenes captured by a handheld cellphone
 - 20 to 62 images, 1/8 views among them for testing



Datasets (Cont'd)

- IMC Phototourism
 - Collection of photos by tourists in a real-world scenario
 - The dataset is provided by the Image Matching Workshop (and Challenge)
 - COLMAP results are available.



Old Town Square Prague, Czech Republic

Credit: Martin Brulalla et al. (2021); <https://nerf-w.github.io/>

Hyperparameters

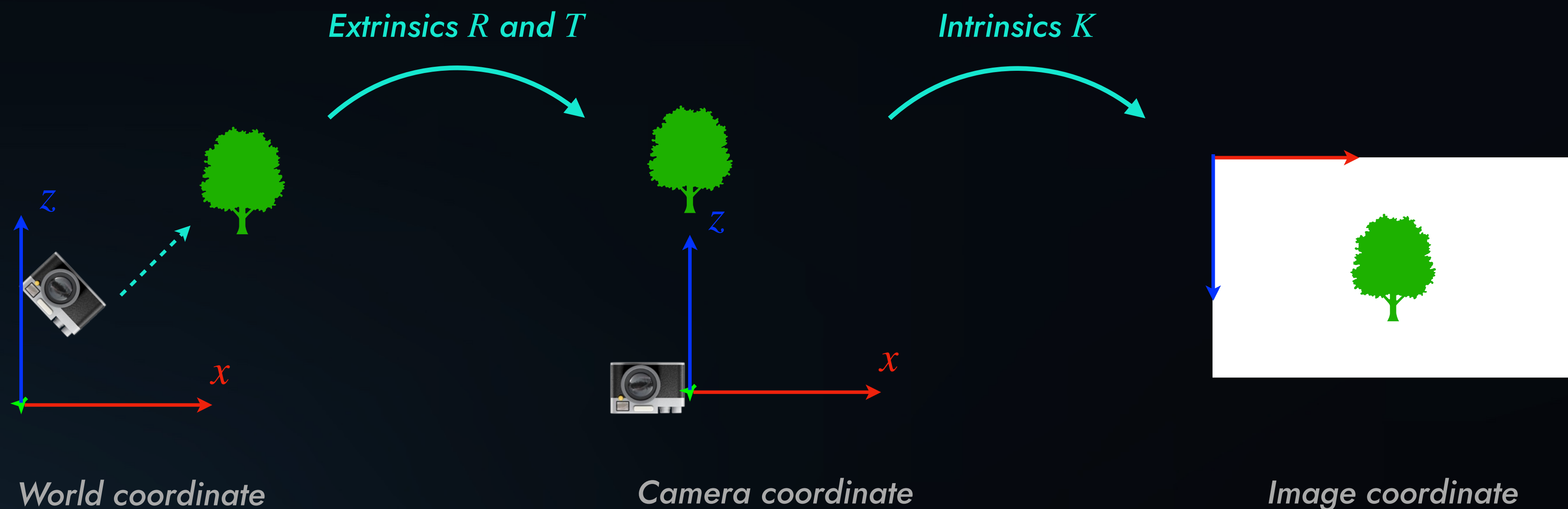
- In the early work, they used a batch size of 4K rays, $N_c=64$, $N_f=128$.
- Adam optimizer with the learning rate of 5×10^{-4} , exponentially decaying to 5×10^{-5} , while the others are set to default.
- Typically, 100-300k iterations to converge on a single NVIDIA V100 GPU taking *1-2 days* (which only takes few minutes nowadays).

Preprocessing

- So, the problem is how to pre-process the dataset?
 - For each image, the camera position $\mathbf{o}$.
 - For each pixel, the viewing direction $\mathbf{d}$ and the corresponding *colors*.
- Terminology & concepts:
 - World, camera, and image coordinate systems
 - Normalized device coordinate (NDC)

Coordinate Transforms

- Extrinsics and intrinsics parameters transform among world, camera, and image coordinates.



Camera projection

$$z_c \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

2D image coordinates *Intrinsic properties (scaling, optic center)* *Extrinsic properties ("inverse" of the camera pose matrix)* *3D world coordinates*

where z_c is the z -coordinate of the camera relative to the world origin. c_x and c_y are the principal point, ideally the center of the image.

- We assume that we have the access to camera pose and intrinsics by COLMAP (Schönberger et al., 2016).
- Usually, we assume the same focal length for x and y , $f = f_x = f_y$.

Extrinsic vs camera pose

- The extrinsic matrix is the inverse of the camera pose matrix.

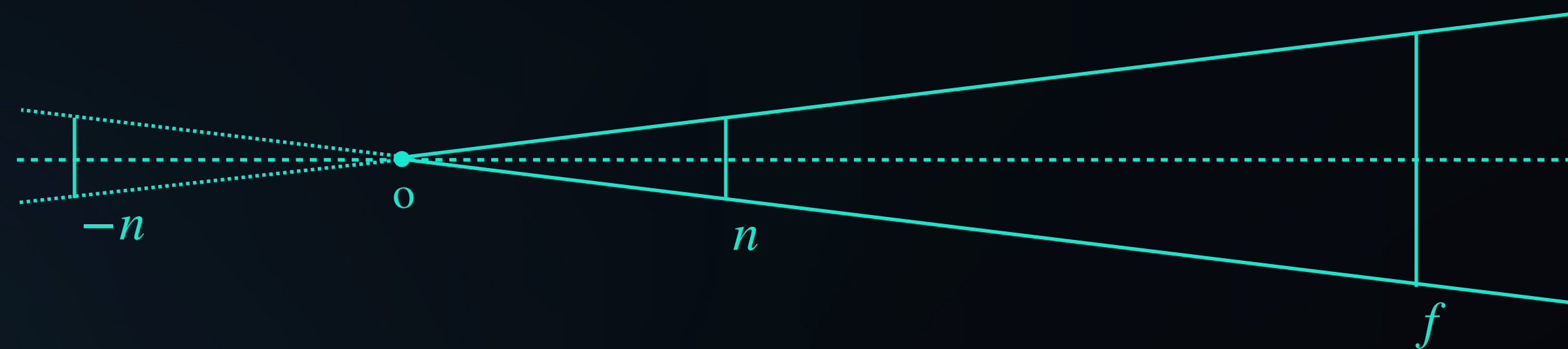
$$\begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_c & \mathbf{C} \\ 0 & 1 \end{pmatrix}^{-1}$$

where $\mathbf{R}_c$ is the *camera rotation matrix* and $\mathbf{C}$ is the *camera position* in the world coordinate system. Note that one may show that:

$$\begin{pmatrix} \mathbf{R}_c & \mathbf{C} \\ 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{R}_c^\top & -\mathbf{R}_c^\top \mathbf{C} \\ 0 & 1 \end{pmatrix}$$

Focal length

- Focal length $\mathbf{f} := (f_x, f_y)$ measures how strongly the system converges light, which is the inverse of the system's *optical power*.
- In our notation, it satisfies $1/f_x = 1/n + 1/f$. But, $1/f$ can be ignored due to $f \gg n$, leading to $f_x = f_y \approx n$, which is the distance to the near plane.
- In our context, the unit of focal length is *pixel*.



Get ray directions

- The ray $r = o + td$ for a pixel in the position of (u, v) of an image can be defined as:

$$d = \left(\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix} \right)^{-1} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \quad o = - \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}^T \begin{pmatrix} t_1 \\ t_2 \\ t_3 \end{pmatrix}$$

*Inverse of intrinsic x extrinsic camera matrix
image space to world space*

*coordinate in
image space*

camera position

- Now, we have preprocessed the dataset! We have d and o , and $C(r)$ is the colors of the pixel in the position of (u, v) in the image

Get ray directions

- The ray $r = o + td$ for a pixel in the position of (u, v) of an image can be defined as:

$$d = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}^{-1} \begin{pmatrix} \frac{u - c_x}{f_x} \\ \frac{v - c_y}{f_y} \\ 1 \end{pmatrix}, \quad o = - \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}^T \begin{pmatrix} t_1 \\ t_2 \\ t_3 \end{pmatrix}$$

*Inverse of the extrinsic rotation matrix
camera space to world space (c2w)*

*direction in
camera space*

camera position

$$\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} 1/f_x & 0 & -c_x/f_x \\ 0 & 1/f_y & -c_y/f_y \\ 0 & 0 & 1 \end{pmatrix}$$

See the block matrix inversion.

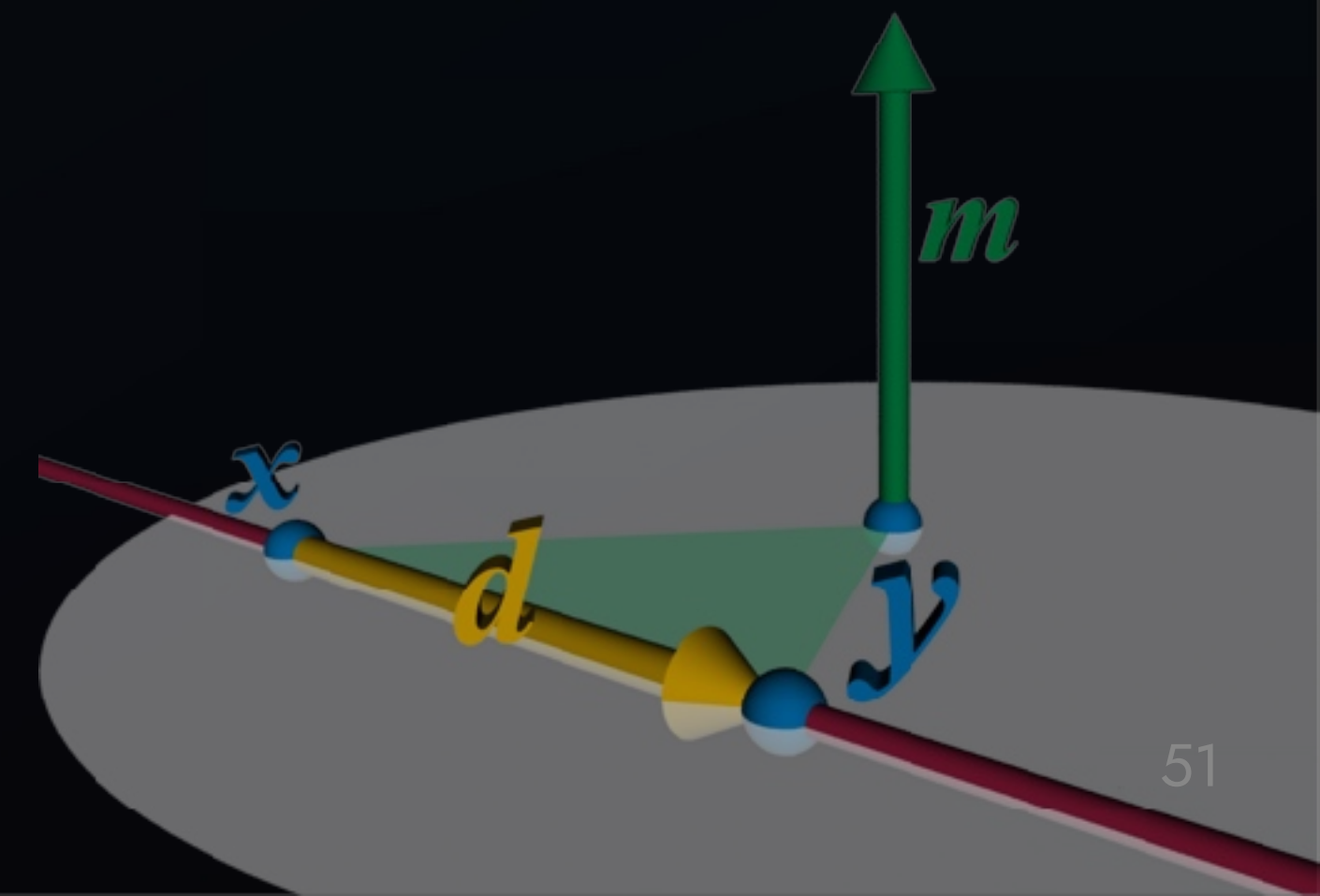
Plücker ray embeddings

- Instead of a typical ray representation $[o; d] \in \mathbb{R}^6$, the Plücker coordinates are as follows:

$$[o \times d; d] \in \mathbb{R}^6.$$

- Where $o \times d$ represents a moment, how far a line is from the origin, and how it is oriented around it. (Lines farther from the origin have larger moments.)
- While $[o; d]$ encodes a ray from a chosen origin, Plücker encodes the line itself. So, line-invariance is more stable for the model to learn.
- *Cons.* They cannot represent a half-line.

In the figure, it's line-invariant, $x \times d = (x + n \cdot d) \times d = y \times d$.



Evaluation metrics

- Peak Signal-to-Noise Ratio (PSNR)
- Structural Similarity Index Map (SSIM)
- Learned Perceptual Image Patch Similarity (LPIPS) ([Zhang et al., 2018](#))

Peak Signal-to-Noise Ratio (PSNR)

- The ratio between the maximum power of a signal and the power of corrupting noise that affects the fidelity of a generated image.
- For the pixels $\in [0,1]$, PSNR is a logarithm of MSE defined as:

$$PSNR = 10 \log_{10} \left(\frac{MAX_I}{MSE} \right)$$
$$MSE = \frac{\sum_{i=1}^H \sum_{j=1}^W (I_0(i,j) - I_1(i,j))^2}{HW}$$

where MAX_I denotes the maximum value of pixel color. For the RGB color space, we take the average over color channel.

Structural Similarity Index Map (SSIM)

- SSIM assesses perceptual differences, luminance, contrast, and structural.

$$l(x, y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}, \quad c(x, y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}, \quad s(x, y) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3}$$

$$\text{SSIM}(x, y) = [l(x, y)]^\alpha [c(x, y)]^\beta [s(x, y)]^\gamma$$

- SSIM can be simplified using $\alpha = \beta = \gamma = 1$, $C_3 = C_2/2$:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

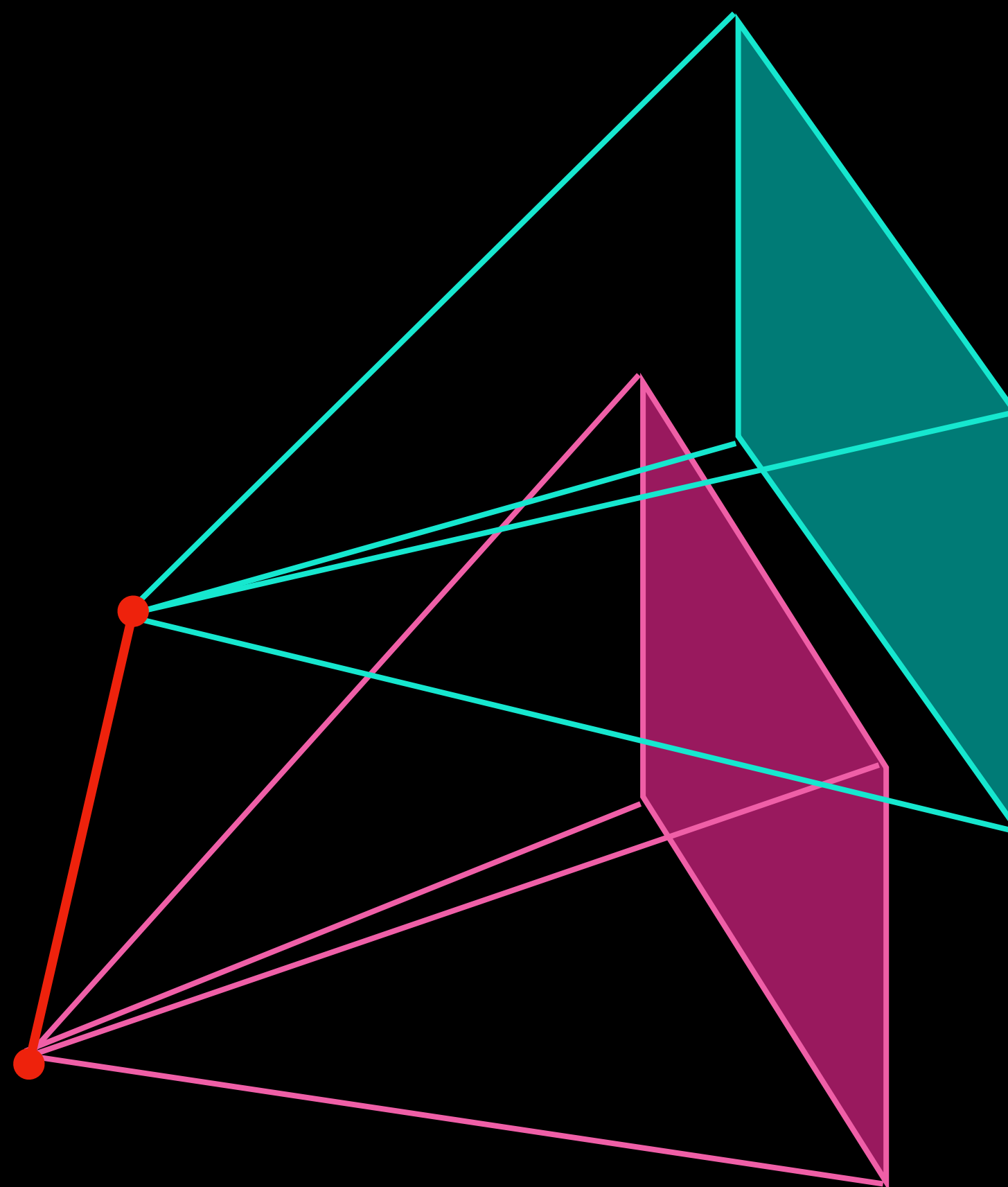
where x, y are local patches, and $C_1 = (k_1L)^2$ and $C_2 = (k_2L)^2$ prevent division by near-zero. L is the max pixel-value. Some* uses $k_1 = 0.01$ and $k_2 = 0.03$, or $C_1 = C_2 = \epsilon$ (a small value).

Learned Perceptual Image Patch Similarity (LPIPS)

- A distance d between two patches x and x_0 , given a network $\mathcal{F}$. The d is calculated using $\hat{y}^l$ and $\hat{y}_0^l$ in $\mathbb{R}^{H_l \times W_l \times C_l}$ for the layer l of $\mathcal{F}$.
- Before that it channel-wisely scales the activations by a vector of $w^l \in \mathbb{R}^{C_l}$, we l2-normalize for the channel dimension. The d is defined as follows:

$$d(x, x_0) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \circ (\hat{y}_{hw}^l - \hat{y}_{0hw}^l)\|_2^2$$

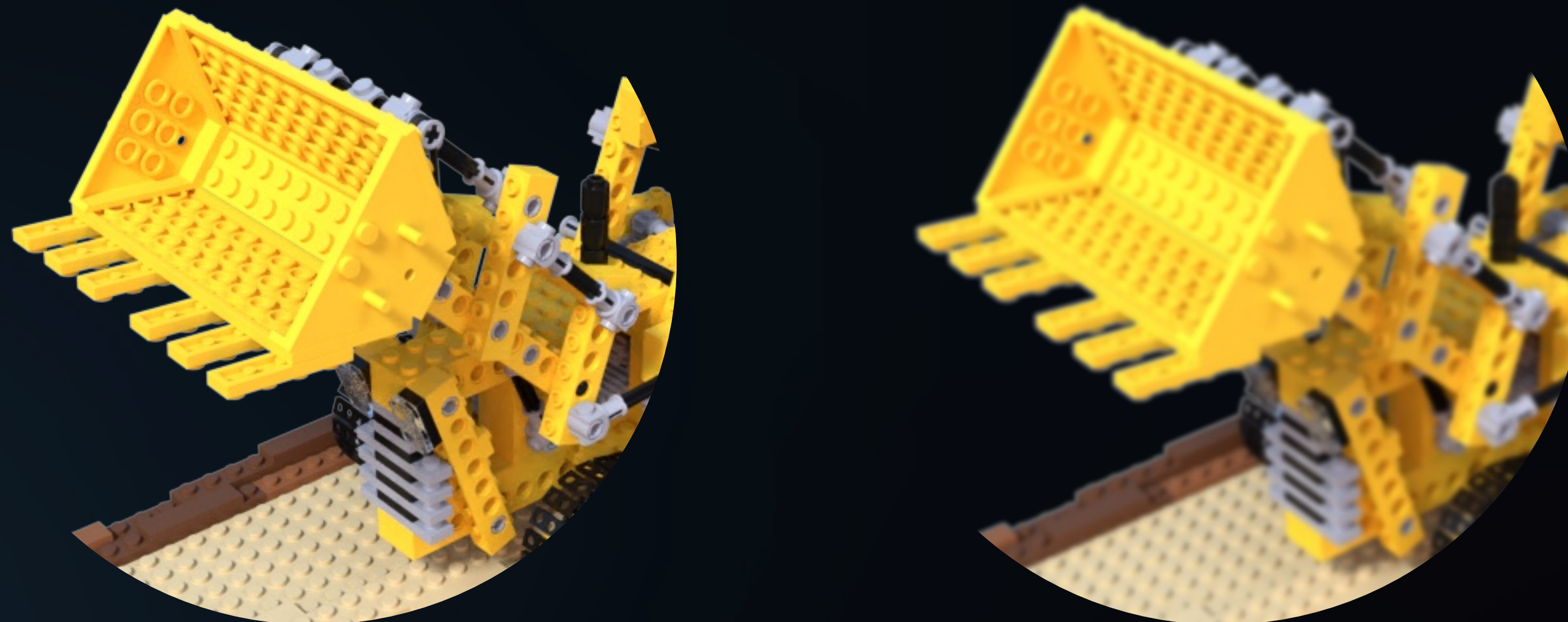
- where the linear weights w_l is fine-tuned ($\mathcal{F}$ is fixed), all fine-tuned, or learning from scratch.



Camera Pose Refinement

Assumption of perfect camera poses

- NeRFs assume that it knows the *perfect* camera poses for training images to get accurate training rays in the world coordinate system for photometric supervision.
- Imperfect camera poses are the source of blurred images or artifactual floatings from inaccurate depth estimation.



Camera pose errors

- Most benchmarks utilizes the COLMAP (SfM) library to estimate camera poses, which used to have some degree of errors.
- COLMAP estimates the camera poses from a set of (un)ordered images. It calculates camera intrinsics and extrinsics (position and orientation).
- With studio capturing, camera calibration is crucial to determine camera intrinsic parameters such as focal length, principal point, and lens distortion coefficients.

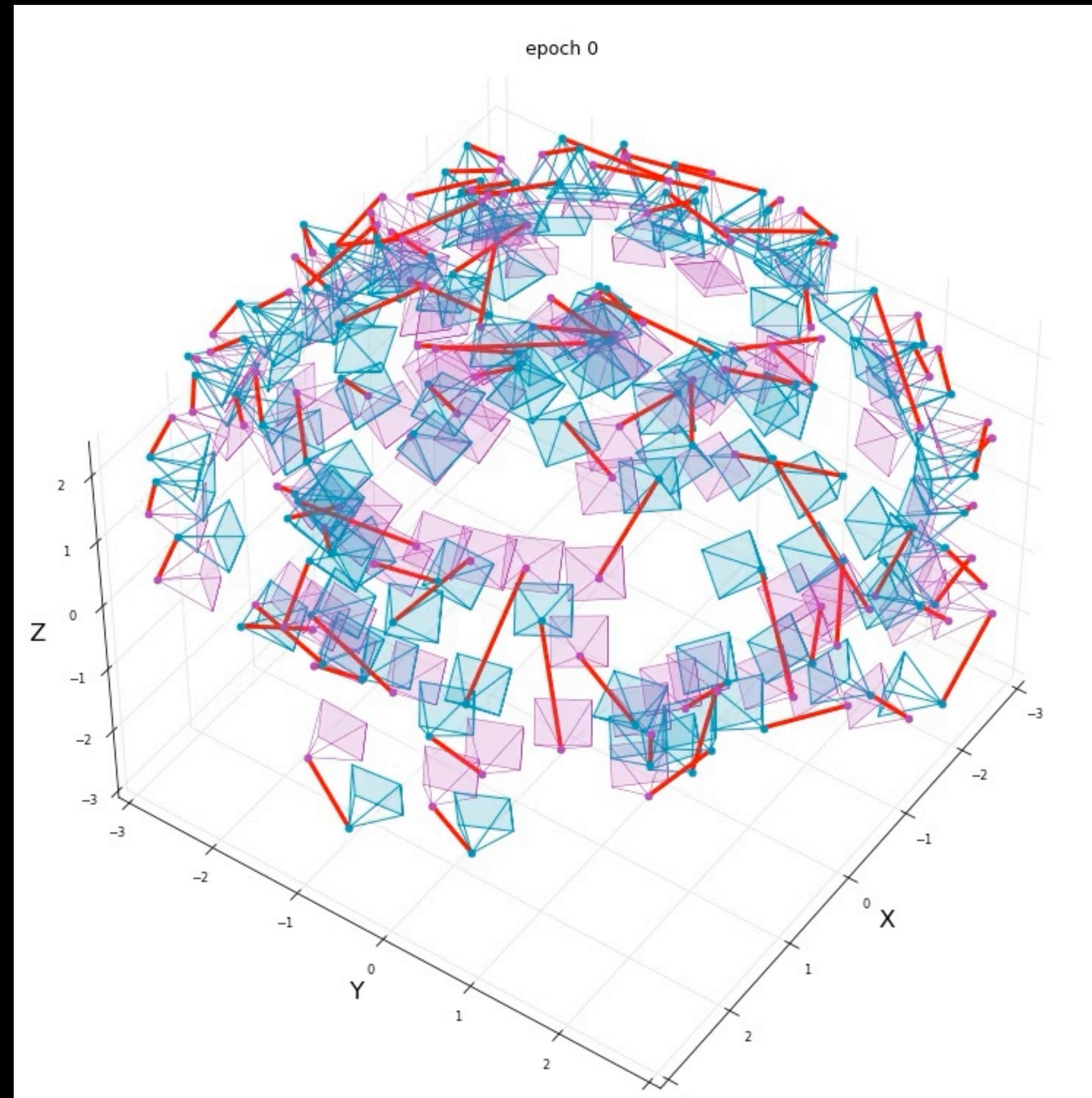
Ray directions with pose errors

- The noised ray $\tilde{r}(t) = \tilde{o} + t\tilde{d}$ for a pixel in the position of (u, v) of an image:

$$\tilde{d} = \underbrace{\begin{pmatrix} \tilde{r}_{11} & \tilde{r}_{12} & \tilde{r}_{13} \\ \tilde{r}_{21} & \tilde{r}_{22} & \tilde{r}_{23} \\ \tilde{r}_{31} & \tilde{r}_{32} & \tilde{r}_{33} \end{pmatrix}^{-1}}_{\text{Inverse of extrinsic rotation matrix}} \underbrace{\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}^{-1}}_{\text{Inverse of intrinsic camera matrix}} \underbrace{\begin{pmatrix} u \\ v \\ 1 \end{pmatrix}}_{\text{coordinate in image space}}, \quad \tilde{o} = - \underbrace{\begin{pmatrix} \tilde{r}_{11} & \tilde{r}_{12} & \tilde{r}_{13} \\ \tilde{r}_{21} & \tilde{r}_{22} & \tilde{r}_{23} \\ \tilde{r}_{31} & \tilde{r}_{32} & \tilde{r}_{33} \end{pmatrix}^{-1}}_{\text{camera position}} \begin{pmatrix} \tilde{t}_1 \\ \tilde{t}_2 \\ \tilde{t}_3 \end{pmatrix}$$

image space to world space

- Here, we assume that the pose errors are tolerable for local refinement methods using error back-propagation. (e.g., $< 60^\circ$)



A major pose error is refined within 5k iterations (not epoch, sorry for that).

Gradient analysis

- The volume rendering with the *stratified sampling* gives the estimated $C(r)$.

$$\hat{C}(r) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) c_i \quad \text{where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

- Let's focus on a unit of photometric supervision, the pair of a ray and its corresponding pixel colors. The gradient with respect to a sample $r(t_i)$ is:

$$\frac{\partial \hat{C}(r)}{\partial r(t_i)} = \frac{\partial \hat{C}(r)}{\partial \sigma_i} \frac{\partial \sigma_i}{\partial r(t_i)} + \frac{\partial \hat{C}(r)}{\partial c_i} \frac{\partial c_i}{\partial r(t_i)}$$

- The network outputs σ_i and c_i are subjects to calculate the input gradient.

Gradient analysis (cont'd)

- In turn, the gradient of a sample $r(t_i)$ on the ray r with respect to $\tilde{o}$ and $\tilde{d}$ is:

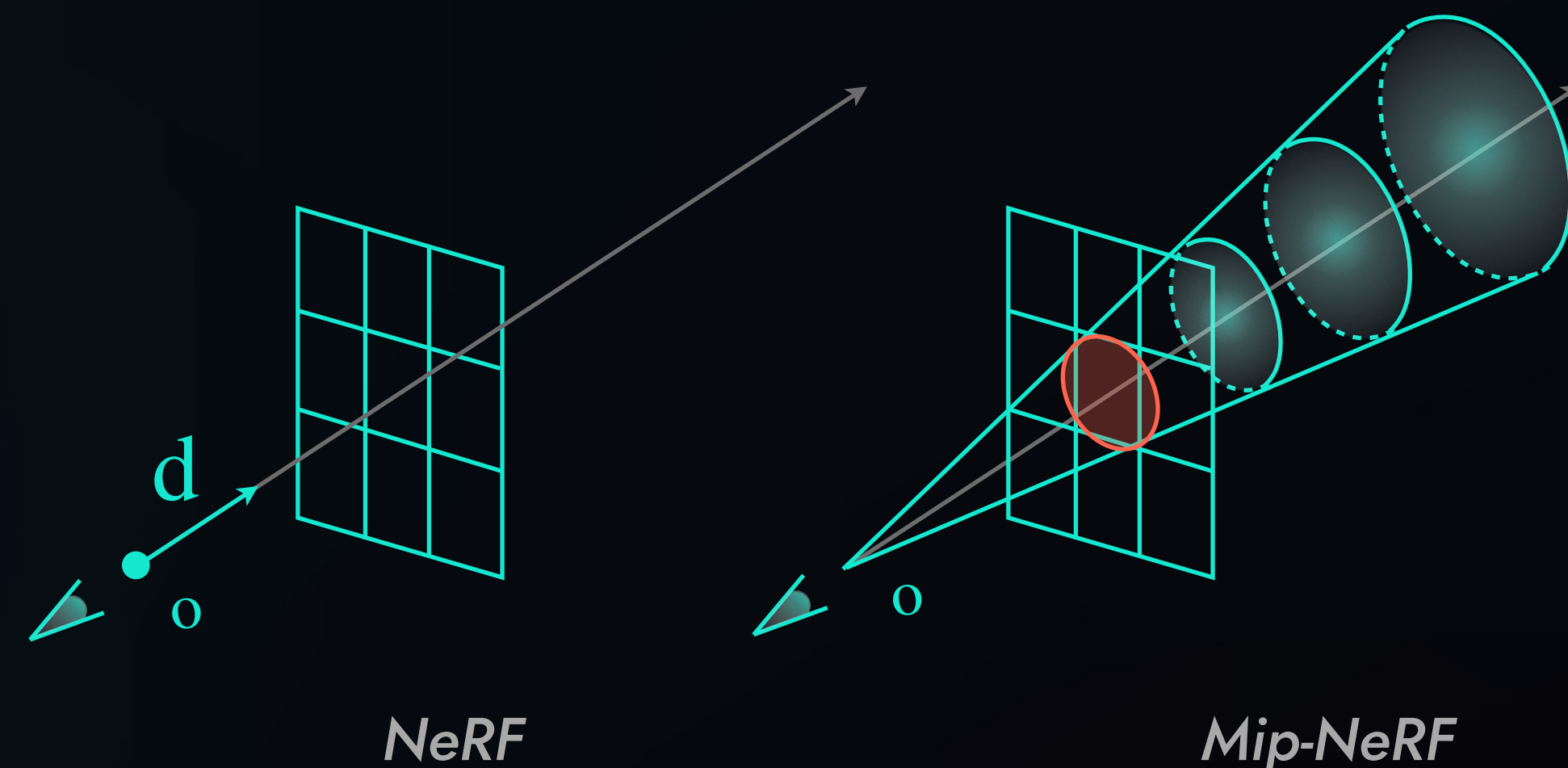
$$\frac{\partial r(t_i)}{\partial \tilde{o}} = \frac{\partial(\tilde{o} + t_i \tilde{d})}{\partial \tilde{o}} = 1$$

$$\frac{\partial r(t_i)}{\partial \tilde{d}} = \frac{\partial(\tilde{o} + t_i \tilde{d})}{\partial \tilde{d}} = t_i$$

- The farther from the ray origin, the more the camera orientation changes.
- Note that the *scale problem* arises between the relative importance of camera location and camera orientation since $E[t_i]$ can be less than 1 or greater than 1.

Recent works may include...

- Voxel-based method
 - Plenoxels (Yu et al., 2021), TensorRF (Chen et al., 2022), Instant-NGP (Müller et al., 2022)
- Camera pose refinement
 - BARF (Lin et al., 2021), GARF (Chng et al., 2022), ✨Instant-Pose (Heo et al., 2023)
 - DUST3R (Wang et al., 2023), MAST3R (Leroy et al., 2024)
- Mip-NeRF (Barron et al., 2021)
- DreamField (Jain et al., 2022)
- DreamFusion (Poole et al., 2022)



Conclusion

- Novel views are synthesized by optimizing the volume rendering function through neural networks, using a sparse set of input images with known poses.
- The whole procedure is differentiable; however, it requires known camera poses to convert image coordinates to world coordinates for training rays.
- We will explore how to learn camera poses through end-to-end optimization, especially for *fast* voxel-based methods, which are known to be challenging.